

Lab 10: Datapath Design(ALU)

Objectives: Datapath Design (ALU)

After completing this lab, you will:

- Learn how to build a simple ALU.

Constructing a Basic Arithmetic Logic Unit

The arithmetic logic unit (ALU) is the brawn of the computer, the device that performs the arithmetic operations like addition and subtraction or logical operations like AND and OR. In this lab, we construct an ALU, in Logisim Evolution, from four hardware building blocks (AND and OR gates, inverters, and multiplexors) and illustrate how combinational logic works.

Because the RISC-V registers are 32 bits wide, we need a 32-bit-wide ALU. Let's assume that we will connect 32 1-bit ALUs to create the desired ALU. We'll therefore start by constructing a 1-bit ALU.

A 1-Bit ALU

The logical operations are easiest, because they map directly onto the hardware components. The 1-bit logical unit for AND and OR looks like Figure 1. The multiplexor on the right then selects a AND b or a OR b, depending on whether the value of Operation is 0 or 1. The line that controls the multiplexor is shown in color to distinguish it from the lines containing data. Notice that we have renamed the control and output lines of the multiplexor to give them names that reflect the function of the ALU. There is no need to build a circuit for these two operations since they will be added as simple gates.

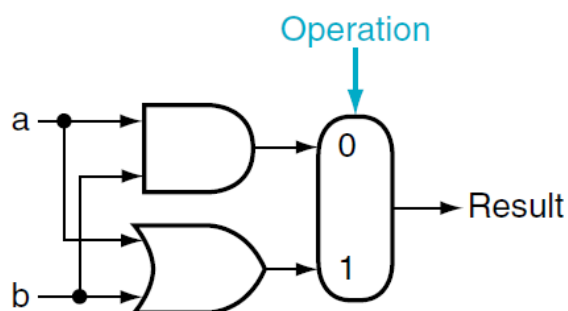


Figure 1: 1-bit logical unit for AND and OR.

The next function to include is addition. An adder must have two inputs for the operands and a single-bit output for the sum. There must be a second output to pass on the carry, called CarryOut. Since the CarryOut from the neighbor adder must be included as an input, we need a third input. This input is called CarryIn. Figure 2 shows the inputs and the outputs of a 1-bit adder. Since we know what addition is supposed to do, we can specify the outputs of this “black box” based on its inputs, as Figure 3 demonstrates. Build this sub-circuit in Logisim-Evolution with the name “OneBitAdder”.

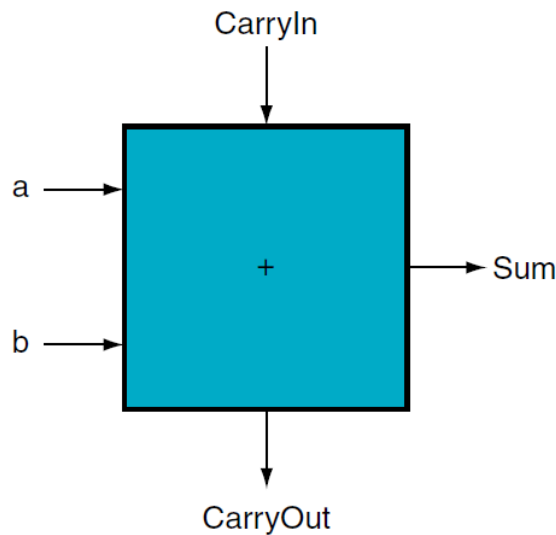


Figure 2: 1-bit Adder.

Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{\text{two}}$
0	0	1	0	1	$0 + 0 + 1 = 01_{\text{two}}$
0	1	0	0	1	$0 + 1 + 0 = 01_{\text{two}}$
0	1	1	1	0	$0 + 1 + 1 = 10_{\text{two}}$
1	0	0	0	1	$1 + 0 + 0 = 01_{\text{two}}$
1	0	1	1	0	$1 + 0 + 1 = 10_{\text{two}}$
1	1	0	1	0	$1 + 1 + 0 = 10_{\text{two}}$
1	1	1	1	1	$1 + 1 + 1 = 11_{\text{two}}$

Figure 3: 1-bit Adder Truth Table.

Figure 4 shows a 1-bit ALU derived by combining the adder with the earlier components. Sometimes designers also want the ALU to perform a few more simple operations, such as generating 0. The easiest way to add an operation is to expand the multiplexor controlled by the Operation line and, for this example, to connect 0 directly to the new input of that expanded multiplexor. Do not build yet, we will still improve it.

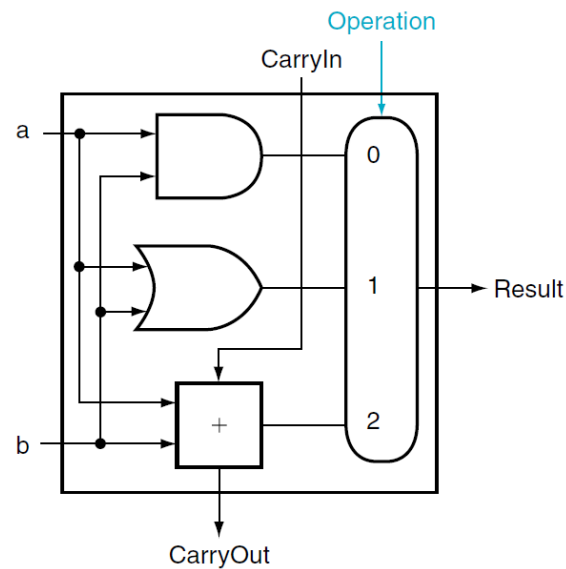


Figure 4: 1-bit ALU.

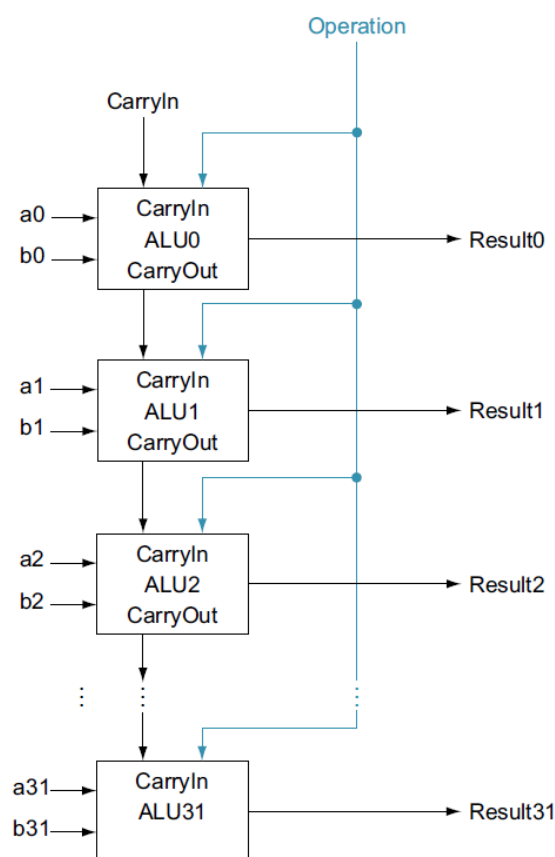


Figure 5: 32-bit ALU.

A 32-Bit ALU

Now that we have completed the 1-bit ALU, the full 32-bit ALU is created by connecting adjacent “black boxes.” Using x_i to mean the i th bit of x , Figure 5 shows a 32-bit ALU. A single carry out of the least significant bit (Result0) can ripple all the way through the adder, causing a carry out of

Subtraction is the same as adding the negative version of an operand, and this is how adders perform subtraction. Recall that the shortcut for negating a two's complement number is to invert each bit (called the one's complement) and then add 1. To invert each bit, we simply add a 2:1 multiplexor that chooses between b and $\neg b$, as Figure 6 shows.

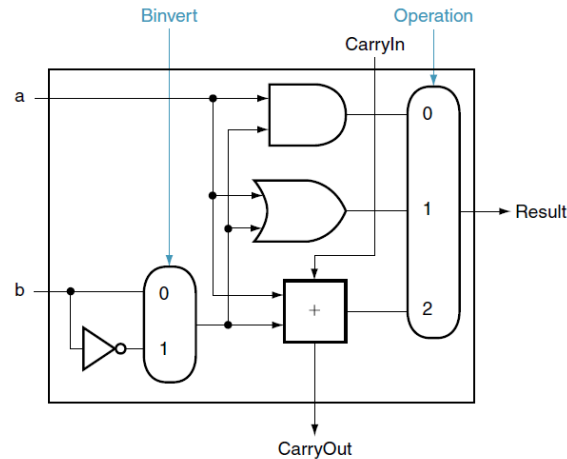


Figure 6: 1-bit ALU with sub.

Suppose we connect 32 of these 1-bit ALUs, as we did in Figure 5. The added multiplexor gives the option of b or its inverted value, depending on Binvert , but this is only one step in negating a two's complement number. Notice that the least significant bit still has a CarryIn signal, even though it's unnecessary for addition. What happens if we set this CarryIn to 1 instead of 0? The adder will then calculate $a + b + 1$. By selecting the inverted version of b , we get exactly what we want:

$$a + \neg b + 1 = a + (\neg b + 1) = a + (-b) = a - b \quad (1)$$

The simplicity of the hardware design of a two's complement adder helps explain why two's complement representation has become the universal standard for integer computer arithmetic. Notice that we could also do that with XORs instead of multiplexors.

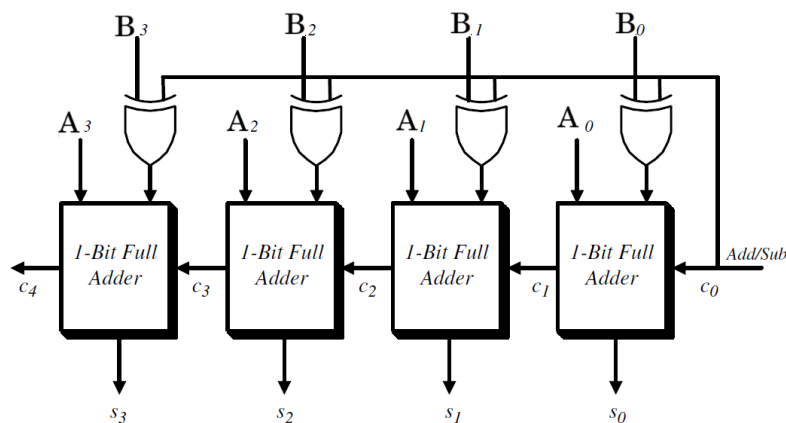


Figure 7: Four Bit Ripple Carry Adder Subtractor.

We also wish to add a NOR function. Instead of adding a separate gate for NOR, we can reuse much of the hardware already in the ALU, like we did for subtract. The insight comes from the following truth about NOR:

$$\neg(a + b) = \neg a \cdot \neg b \quad (2)$$

Since we have AND and NOT b, we only need to add NOT a to the ALU. Figure 8 shows that change.

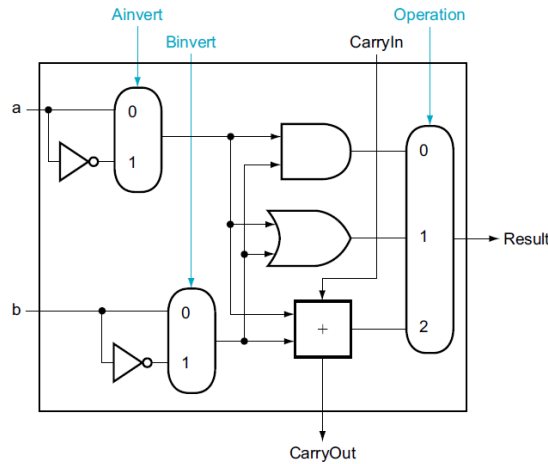


Figure 8: 1-bit ALU with NOR.

Tailoring the 32-Bit ALU to RISC-V

These four operations—add, subtract, AND, OR—are found in the ALU of almost every computer, and the operations of most RISC-V instructions can be performed by this ALU. But the design of the ALU is incomplete.

One instruction that still needs support is the set less than instruction (*slt*). Recall that the operation produces 1 if *rs1* < *rs2*, and 0 otherwise. Consequently, *slt* will set all but the least significant bit to 0, with the least significant bit set according to the comparison. For the ALU to perform *slt*, we first need to expand the three-input multiplexor in Figure 8 to add an input for the *slt* result. We call that new input *Less* and use it only for *slt*.

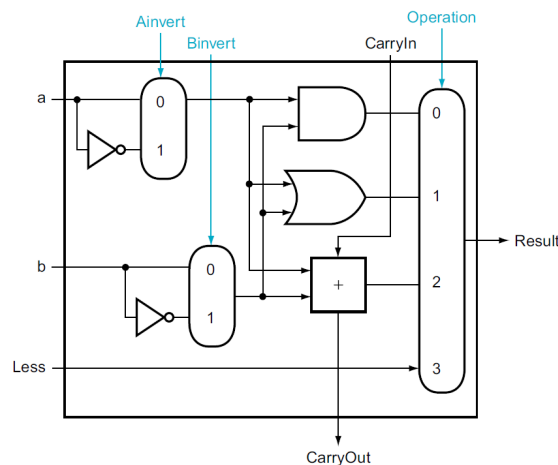


Figure 9: 1-bit ALU with Less.

Figure 9 shows the new 1-bit ALU with the expanded multiplexor. From the description of slt above, we must connect 0 to the Less input for the upper 31 bits of the ALU, since those bits are always set to 0. What remains to consider is how to compare and set the least significant bit for set less than instructions.

What happens if we subtract b from a ? If the difference is negative, then $a < b$. We want the least significant bit of a set less than operation to be a 1 if $a < b$; that is, a 1 if $a - b$ is negative and a 0 if it's positive. This desired result corresponds exactly to the sign bit values: 1 means negative and 0 means positive. Following this line of argument, we need only connect the sign bit from the adder output to the least significant bit to get set less than. (Alas, this argument only holds if the subtraction does not overflow; we will explore a complete implementation later.)

Unfortunately, the Result output from the most significant ALU bit in Figure 9 for the slt operation is not the output of the adder; the ALU output for the slt operation is obviously the input value Less.

Thus, we need a new 1-bit ALU for the most significant bit that has an extra output bit: the adder output. Figure 10 shows the design, with this new adder output line called Set. As long as we need a special ALU for the most significant bit, we added the overflow detection logic since it is also associated with that bit. Figure 11 shows the 32-bit ALU.

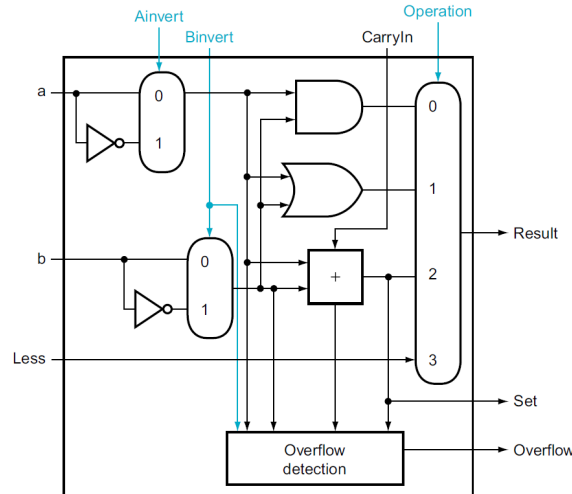


Figure 10: 1-bit ALU with Set output.

Notice that every time we want the ALU to subtract, we set both CarryIn and Binvert to 1. For adds or logical operations, we want both control lines to be 0. We can therefore simplify control of the ALU by combining the CarryIn and Binvert to a single control line called Bnegate.

To further tailor the ALU to the RISC-V instruction set, we must support conditional branch instructions such as Branch if Equal (beq), which branches if two registers are equal. The easiest way to test equality with the ALU is to subtract b from a and then test to see if the result is 0. Thus, if we add hardware to test if the result is 0, we can test for equality. The simplest way is to OR all the outputs together and then send that signal through an inverter:

$$Zero = \neg(\text{Result}_{31} + \text{Result}_{30} + \dots + \text{Result}_1 + \text{Result}_0) \quad (3)$$

Figure 13 shows the revised 32-bit ALU. We can think of the combination of the 1-bit Ainvert line, the 1-bit Bnegate line, and the 2-bit Operation lines as 4-bit control lines for the ALU, telling it to perform add, subtract, AND, OR, NOR, or set less than. Figure 12 shows the ALU control lines and the corresponding ALU operation.

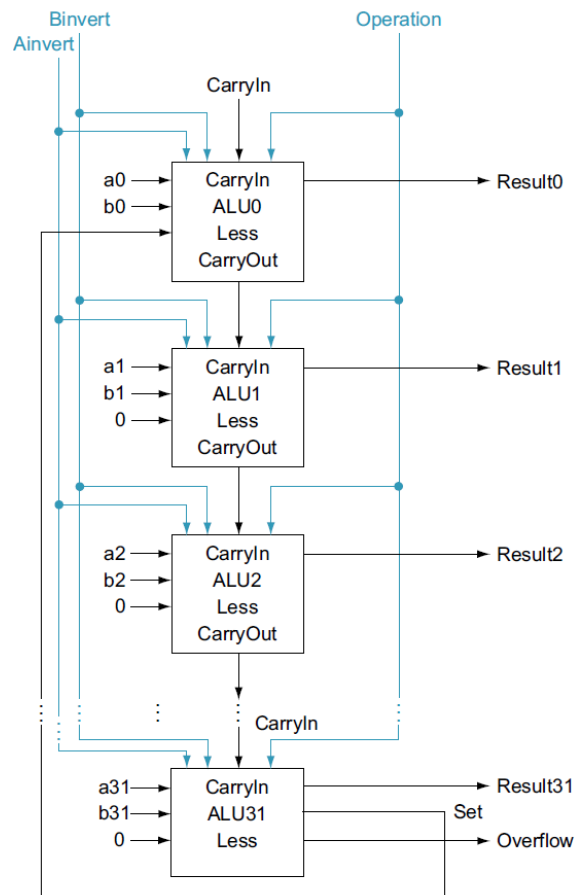


Figure 11: 32-bit ALU improved.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

Figure 12: ALU Control.

Assignment

1. Build the one-bit adder sub-circuit (Figure 2).
2. Build the one-bit ALU sub-circuit, named "OneBitALU", using the adder sub-circuit (Figure 10).
3. Build a 32-bit ALU (Figure 13) using the "OneBitALU" sub-circuits.
4. **Homework: Complete the overflow logic and modify the "OneBitALU" sub-circuits so that slt will work with and without overflow.**

Figure 13: 32-bit ALU with zero signal.